air-classifier — SERVICE SPECIFICATION

IN THIS REPO · +PROD PRODUCTION REQUIREMENT · EXT OTHER PROJECT / MANAGED

GKE CLUSTER — EAST-WEST TRAFFIC ONLY · NO INTERNET EXPOSURE · NETWORKPOLICY RESTRICTS CALLERS BY NAMESPACE

PEER MICROSERVICES

review ingest · feedback pipelines · CRM & support tooling — bulk, latency-tolerant

auth: X-Api-Key per service
issued & rotated by air-infra

air-platform

chat entry point for external applications · its classifier client replaces the old in-core intent module

auth: X-Api-Key
deadline 80ms · fail-open

ONE API, TWO CLASSES

Both consumers call the same routes with the same API-key scheme. The key identifies the caller for quota, priority class and metering — it is not a trust boundary on its own. NetworkPolicy plus per-key scopes do that work.

air-classifier — OWN REPO · OWN DEPLOYABLE · STATELESS REPLICAS

Sentiment · aspect sentiment · intent · tone · urgency · abuse · spam & fake-review · language · PII presence. One model set, two traffic shapes — so two admission classes and two replica pools behind a single API.

BATCH ROUTES — peer services

```
POST /v1/classify/batch → { job_id }
GET  /v1/jobs/{id} → status · progress · results
DELETE /v1/jobs/{id} → cancel + purge
```

polling only — no callbacks to maintain

SYNC ROUTES — air-platform + peers

```
POST /v1/classify → verdict (single text)
GET  /v1/taxonomy → label contract + version
GET  /healthz · /readyz · /metrics
```

priority=interactive reserved for air-platform

ADMISSION CONTROL +PROD

what makes one service safe for two consumers

- API key → tenant, scopes, priority class, quota
- Interactive preempts batch, always
- Separate replica pools, not just priorities — a batch flood cannot consume interactive capacity
- Load shedding: batch is deferred before any interactive request queues
- Per-call usage events for metering and cost attribution

INTERACTIVE POOL

sync · p95 < 60ms · models warm in memory · no queueing, shed rather than stall · HPA on inflight requests

BATCH POOL

queue-backed jobs · chunked with resumable progress · large micro-batches for throughput · HPA on queue depth

SHARED INFERENCE PIPELINE

normalize · language detect · PII scrub → cache lookup — hit returns here → model ensemble — one head per attribute → calibrate · per-class threshold → map to taxonomy · stamp model_version + taxonomy_version

CACHE KEY CONTRACT +PROD

sha256(normalized_text) + model_version + taxonomy_version + tenant_id — version components are mandatory, or a model upgrade silently serves stale verdicts. TTL by attribute class: stable labels days, adversarial labels (spam, fake review) minutes.

DEPENDENCIES

LOCAL MODEL SET — in-process
small multilingual transformers, one head per attribute · loaded at startup, version pinned in config · owns the 60ms budget

air-llm EXT
model gateway — local model or cloud provider behind one API · used here for zero-shot labels with no trained head yet and new-taxonomy bootstrapping · batch pool only, never the interactive path

REDIS EXT via air-infra
verdict cache. Ecommerce text repeats heavily — template reviews, canned support replies, duplicate feedback across channels — so hit rate is the main cost and latency lever. Track hit rate per attribute as a first-class metric

POSTGRES EXT via air-infra
housekeeping and extended features: batch job state and chunk progress · API-key registry and quota counters · usage ledger for metering · label-feedback table (corrections that feed the next training set) · retention jobs. Service owns its schema; air-infra owns provisioning, backup and migration tooling

OBJECT STORE EXT via air-infra
large batch result sets behind a signed URL, with a TTL · keeps result payloads out of Postgres

TRAINING & EVAL — offline
labelled sets and per-label precision/recall live in evals/ · a model, threshold or taxonomy change ships only on a passing suite · pipeline runs outside this deployable

/v1/taxonomy — WHY THE SERVICE PUBLISHES ITS OWN LABEL CONTRACT

The taxonomy is the machine-readable description of everything this service can return: each attribute, its permitted label values, its score range, and the version of the set as a whole. **Publishing it as an endpoint means no consumer hardcodes a copy.**

- **air-platform** validates its routing rules against it at startup — a rule referencing a retired label fails on deploy, not mid-conversation
- **Peer services** render, filter and aggregate results without shipping their own label list, so a new label reaches dashboards without a client release
- **Every verdict carries taxonomy_version**, so a consumer can pin a version and detect drift when the service upgrades
- **Label changes ship as config** — the set is versioned data in this repo, not constants compiled into six codebases
- **SDD**: the taxonomy file is the generation source for typed enums and validators on both sides of the wire

Compatibility rule: additive labels only within a major version; removing or re-meaning a label is a major bump with a published deprecation window and both versions servable during it. Retraining that shifts score distributions is a breaking change even when the schema is untouched — announce it the same way.

ECOMMERCE LABEL SET — v1

sentiment	positive · neutral · negative · mixed (+ score -1..1)
aspect_sentiment	shipping · product_quality · sizing · price · packaging · support · returns → per-aspect polarity + confidence
intent	complaint · return_request · refund_request · product_question · shipping_query · praise · cancellation · fraud_report · other
tone	angry · frustrated · neutral · satisfied · delighted
urgency	low · medium · high · critical
abuse	toxicity · profanity · threat (scores)
integrity	spam · incentivized · bot_generated · rating_text_mismatch (scores)
language	ISO 639-1 + confidence
pii_present	boolean + character spans

Document-level sentiment alone is not enough for this domain — "fast delivery, terrible fabric" is the normal case, and only aspect sentiment preserves that signal for merchandising and supplier decisions.

PRODUCTION REVIEW — GENERAL-PURPOSE CLASSIFIER IN AN ECOMMERCE DOMAIN

01 · MULTILINGUAL AND CODE-MIXED BY DEFAULT

Marketplace text arrives mixed-language, transliterated and emoji-heavy. One multilingual model beats a per-language fleet on both accuracy and ops cost, but only if you measure per language — a strong global F1 routinely hides a weak long tail. Report metrics per language and per tenant, never as one number.

02 · CALIBRATION OVER ACCURACY

Review data is heavily positive-skewed, so accuracy flatters a useless model. Track precision and recall per label, set thresholds per class rather than one global cut, and calibrate scores so a 0.8 means the same thing across attributes — downstream services will compare them whether or not you intend it.

03 · DRIFT IS THE STEADY-STATE RISK

New categories, seasonal peaks and changing slang move the input distribution continuously. Monitor score distributions per tenant and alert on shift; re-evaluate on a rolling labelled sample on a schedule, not only at release. The label-feedback table is what makes each retraining round cheaper than the last.

04 · PII AND RETENTION

Reviews and support text carry names, addresses and order numbers. Scrub before caching, storing or logging — never after. Results get a TTL, jobs get a purge path, and a per-tenant delete request must reach cache, Postgres and object store together. Design this now; retrofitting deletion across three stores is painful.

05 · INTEGRITY LABELS ARE ADVERSARIAL

Spam and fake-review detection has an opponent that adapts; sentiment does not. Retrain those heads on a faster cadence, cache their verdicts for minutes rather than days, and keep the scores internal — publishing a fake-review score to the party generating it hands them a test harness.

06 · COST IS A CACHE PROBLEM

At batch volumes, unit cost is set by hit rate far more than by model size. Instrument cost per thousand classifications and hit rate per attribute from day one, and normalize text aggressively before hashing — case, whitespace, punctuation and emoji folding turn near-duplicates into cache hits.

air-classifier REPOSITORY — SPEC-DRIVEN LAYOUT

spec/ is the source of truth: the API, the taxonomy and the SLOs are authored there and everything else is generated from or validated against them.

LAYOUT

```
air-classifier/
├ spec/                    SDD source of truth
│  ├ openapi.yaml          all routes + schemas
│  ├ taxonomy/v1.yaml      label contract
│  ├ slo.yaml              latency + availability
│  └ adr/                  decisions, dated
├ src/air_classifier/
│  ├ api/                  handlers (generated)
│  ├ admission/            keys · quotas · priority
│  ├ pipeline/             normalize → cache →
│  │                       ensemble → calibrate → map
│  ├ models/               loaders, version pinning
│  ├ cache/                redis keying + TTL policy
│  ├ store/                jobs · usage · feedback
│  └ clients/              air-llm
├ evals/                   labelled sets, per-label
│                          metrics, drift checks
├ tests/
├ deploy/                  manifests · HPA ×2 pools
│                          NetworkPolicy · migrations
└ Dockerfile
```

WHAT GENERATES FROM SPEC

Request and response models, route stubs and validators from openapi.yaml. Typed label enums and score bounds from taxonomy/v1.yaml, published as a small client package that air-platform and peer services consume. Alert rules and dashboards from slo.yaml. Regenerate in CI and fail the build on drift, so hand-edits to generated files cannot survive.

SLOs TO AUTHOR FIRST

Interactive p95 < 60ms and p99 < 120ms at the service boundary; 99.9% availability on sync routes; batch throughput per pod and a job completion target by size band; cache hit rate floor. These numbers drive pool sizing, HPA targets and the shed threshold, so they belong in spec/ before any handler is written.

FOUR PROJECTS, FOUR REPOS

air-platform · air-classifier · air-llm · air-infra. Separate repos give each service its own cadence and owner, and they push one obligation up front: the shared contracts have to live somewhere both sides trust. Publish each service's spec artifacts as a versioned client package from its own repo, and consume by pinned version — the alternative, copied schema files, drifts within weeks.

BUILD ORDER

1 — spec/ complete: openapi, taxonomy v1, SLOs
2 — sync path: /v1/classify + /v1/taxonomy, one model head, no cache
3 — Redis cache with the full key contract, hit-rate metrics
4 — batch path: jobs, chunking, object-store results
5 — admission control, quotas, metering
6 — remaining heads, evals in CI, drift monitors